INTELLIGENT JOB APPLICATION AUTOMATION PLATFORM

Product Requirements Document (PRD) • Single-Portal MVP

AI-assisted job discovery, relevance analysis, form understanding,
candidate-data mapping, intelligent question answering, and application submission

Document Property	Value
Document type	Product Requirements Document
Scope	Single job portal, end-to-end MVP
Primary users	Staffing / recruitment operations team
Target market	USA job opportunities and candidate submissions
Primary automation	Browser-based application workflow
Recommended stack	TypeScript/Node.js + Playwright + PostgreSQL + LLM
Version	MVP v1.0

Core principle: AI decides. The automation engine executes. Humans handle exceptions.

1. Executive Summary

This project proposes an Intelligent Job Application Automation Platform designed to reduce repetitive manual effort in submitting candidates to relevant job opportunities on one selected job portal. The first release deliberately focuses on a single portal. This constraint is intentional: it allows the team to solve browser automation, application-form understanding, candidate-data mapping, AI-assisted decisions, and recovery behavior deeply before expanding to other portals.

The system accepts a candidate profile and resume, a target role, and search criteria such as location and experience. It navigates the selected job portal, retrieves candidate-relevant jobs, evaluates job fit, starts an application for a selected job, inspects the application page, and converts web controls into structured form objects. Known fields are filled from trusted candidate data. Context-dependent questions are passed to an AI decision layer with candidate facts, job context, question text, and the available options. The AI returns a structured decision, not a browser command. The browser automation layer validates that decision and performs the corresponding UI action.

The MVP is designed to fail safely. CAPTCHA, MFA, blocked access, unknown form widgets, missing candidate facts, low-confidence AI answers, or sensitive questions should create a human-review state rather than forcing the system to guess. The product is therefore an intelligent assistant with controlled autonomy, not a blind macro.

1.1 Product Vision

Build a reliable, explainable automation assistant that behaves like a careful recruitment operations specialist: fast on repetitive work, grounded in candidate facts, explicit about uncertainty, and auditable after every application.

1.2 MVP Scope

- One job portal only.
- USA-focused job search criteria such as role, location, experience, skills, and work mode.
- Candidate profile creation from structured input plus resume parsing.
- Job extraction, deduplication, and relevance scoring.
- Application form discovery from the DOM and normalization into structured fields.
- Deterministic mapping for known profile fields such as name, email, phone, education, experience, and resume upload.
- AI-assisted handling of supported free-text, radio, checkbox, and select questions.
- Pre-submission validation and human review for uncertain or high-impact cases.
- Application logging, error reporting, and execution history.
- No attempt to bypass CAPTCHA, MFA, bot protection, rate limits, or access controls.

1.3 Out of Scope

- Multi-portal support.
- CAPTCHA solving or security-control bypass.
- Fully autonomous handling of every imaginable question or custom widget.
- Unsupervised decisions when required candidate facts are missing.
- Mass parallel application submission at scale.
- Guaranteed compatibility with arbitrary portal redesigns.

2. Problem Statement and Business Need

Staffing operations often require the same candidate information to be entered into multiple job application workflows. The work is repetitive, time-consuming, difficult to standardize, and prone to mistakes. A recruiter may spend significant time searching for jobs, reading job descriptions, determining whether a job is relevant, navigating an application, copying candidate details, answering screening questions, uploading documents, and recording the outcome.

The proposed platform addresses this bottleneck by decomposing the workflow into machine-friendly layers: job discovery, relevance analysis, candidate knowledge, DOM/form understanding, AI decision-making, browser execution, validation, and tracking.

2.1 Current Manual Workflow

1. Search for a target role and location.
2. Review a large number of jobs manually.
3. Open potentially relevant jobs and compare requirements with the candidate.
4. Start the application and navigate through multiple pages.
5. Copy candidate information from a resume, spreadsheet, or internal system.
6. Answer repeated screening questions manually.
7. Interpret new questions that appear only on particular jobs.
8. Upload the resume and complete required fields.
9. Submit the application and manually record the result.

2.2 Pain Points

Pain Point	Operational Impact	Automation Opportunity
Repetitive data entry	High time consumption	Profile-to-field mapping
Job screening	Human time spent on low-fit jobs	Rule + semantic relevance scoring
Question variation	Slow manual interpretation	AI-assisted structured answers
Inconsistent answers	Candidate-profile mismatch risk	Central candidate knowledge base
Missed required fields	Incomplete submissions	Form validation engine
Poor traceability	Difficult auditing	Application logs and decision history

3. Objectives, Success Criteria, and Principles

3.1 Product Objectives

- Reduce manual effort per application without sacrificing candidate-data accuracy.
- Maintain one structured candidate knowledge source for consistent answers.
- Understand dynamic application questions using context-aware AI.
- Make automated decisions traceable to facts, rules, or model output.
- Create reusable core services while keeping the first portal integration isolated.
- Fail safely when uncertainty, security controls, or unsupported UI states appear.

3.2 Suggested MVP Success Criteria

Metric	Target
Relevant-job precision	≥ 85% on a curated validation set
Known-field mapping accuracy	≥ 95% for supported fields
Supported form-control detection	≥ 90%
AI answer validity	≥ 90% on approved benchmark; ambiguous cases excluded
Eligible application completion	≥ 80% in controlled tests
Auditability	100% of automated applications have durable logs
Duplicate submissions	0 in regression testing

3.3 Engineering Principles

- Deterministic before generative.
- AI chooses semantics; code controls actions.
- Never trust raw model output without validation.
- Use human review for uncertainty and high-impact decisions.
- Keep portal-specific behavior in an adapter.
- Treat observability as part of the product.

4. Functional Requirements

4.1 Candidate Management

- Create and update candidate profiles.
- Upload and parse resumes.
- Store normalized candidate attributes in structured form.
- Allow an operator to correct extracted data before automation.
- Store job-search preferences and target roles.
- Track candidate-specific documents and versions.

4.2 Job Search and Discovery

- Open the selected portal and execute configured searches.
- Apply supported filters such as role, location, experience, work mode, and employment type.
- Extract job title, company, location, description, requirements, URL, and source metadata.
- Deduplicate jobs.
- Persist discovered jobs and search timestamps.

4.3 Job Relevance Analysis

The relevance engine should combine hard filters with semantic comparison. Hard filters remove clearly unsuitable jobs. The scoring layer then evaluates role, skills, experience, location, education, and other requirements. The result should contain a score, major matches, gaps, and a recommendation.

```
Job Relevance Pipeline
```

```
1. Hard filters: location, role family, experience constraints
2. Extract entities: skills, degree, tools, certifications
3. Semantic comparison: candidate vs. job description
4. Normalize score: 0-100
5. Decision: APPLY / REVIEW / SKIP
6. Store reason codes and evidence
```

4.4 Application Workflow

10. Open the selected job.
11. Verify that the application path is supported.
12. Start the application.
13. Inspect the current page.
14. Normalize detected form controls.
15. Resolve known fields from trusted candidate data.
16. Send unresolved/context-dependent questions to the AI decision layer.
17. Validate the AI decision against the allowed options and candidate facts.
18. Execute UI actions.
19. Validate required fields and profile consistency.
20. Submit where permitted and only after validation.
21. Record the outcome.

4.5 MVP-Supported Controls

Control	Example	Handling
Text input	First name	Direct profile mapping
Email	Email address	Mapping + validation
Phone	Phone number	Mapping + format validation
Textarea	Why interested?	AI-generated response with limits
Select	Highest education	Rules/AI select an allowed option
Radio	Work authorization	Rules/AI select an allowed option
Checkbox	Skills/declarations	Rules/AI map selections
File upload	Resume	Attach configured candidate document
Button	Next / Submit	Deterministic navigation

5. Proposed System Architecture

The recommended MVP uses Node.js with TypeScript as the primary backend runtime and Playwright as the browser automation layer. Python can be introduced as a supporting service later for specialized document or NLP workloads, but the MVP should avoid unnecessary runtime fragmentation.

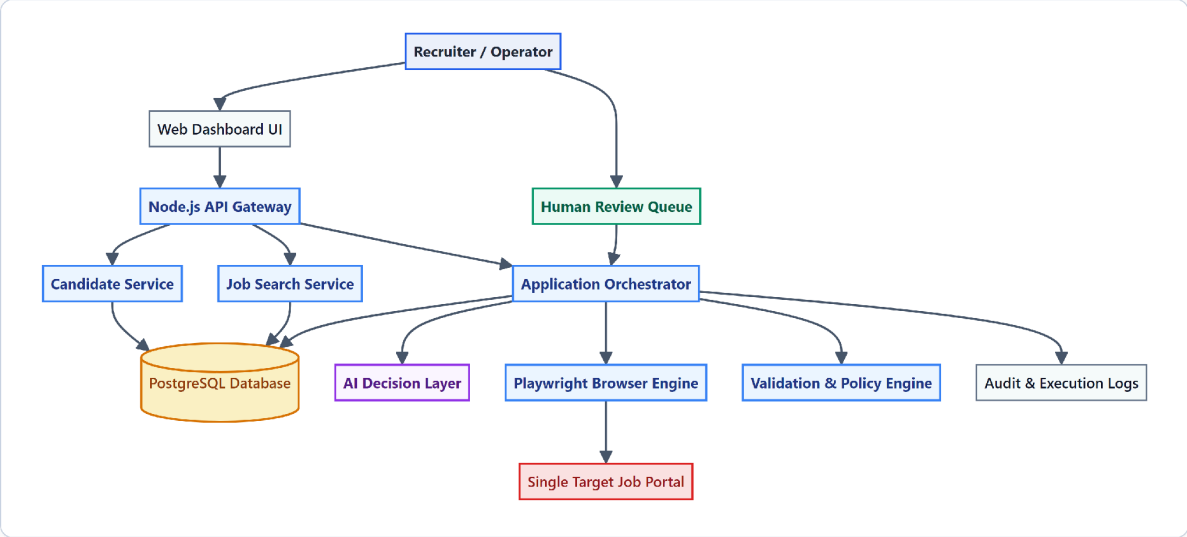


Figure 1: Proposed MVP System Architecture & Component Interactions

5.1 Core Components

Component	Responsibility
Dashboard	Candidate setup, job search, application status, review queue
Node.js API	Auth, orchestration, configuration, workflow endpoints
Candidate Service	Profile and resume-derived facts
Job Search Service	Search, extraction, deduplication
Relevance Engine	Rules, scoring, recommendation
Application Orchestrator	Controls application state machine
DOM/Form Engine	Detects and normalizes page controls
AI Decision Layer	Interprets questions and chooses structured answers
Validation/Policy	Checks options, confidence, required fields, safety
Browser Automation	Clicks, typing, uploads, navigation
Audit/Logging	Events, decisions, screenshots/trace references
PostgreSQL	Persistent data store

5.2 Application State Machine

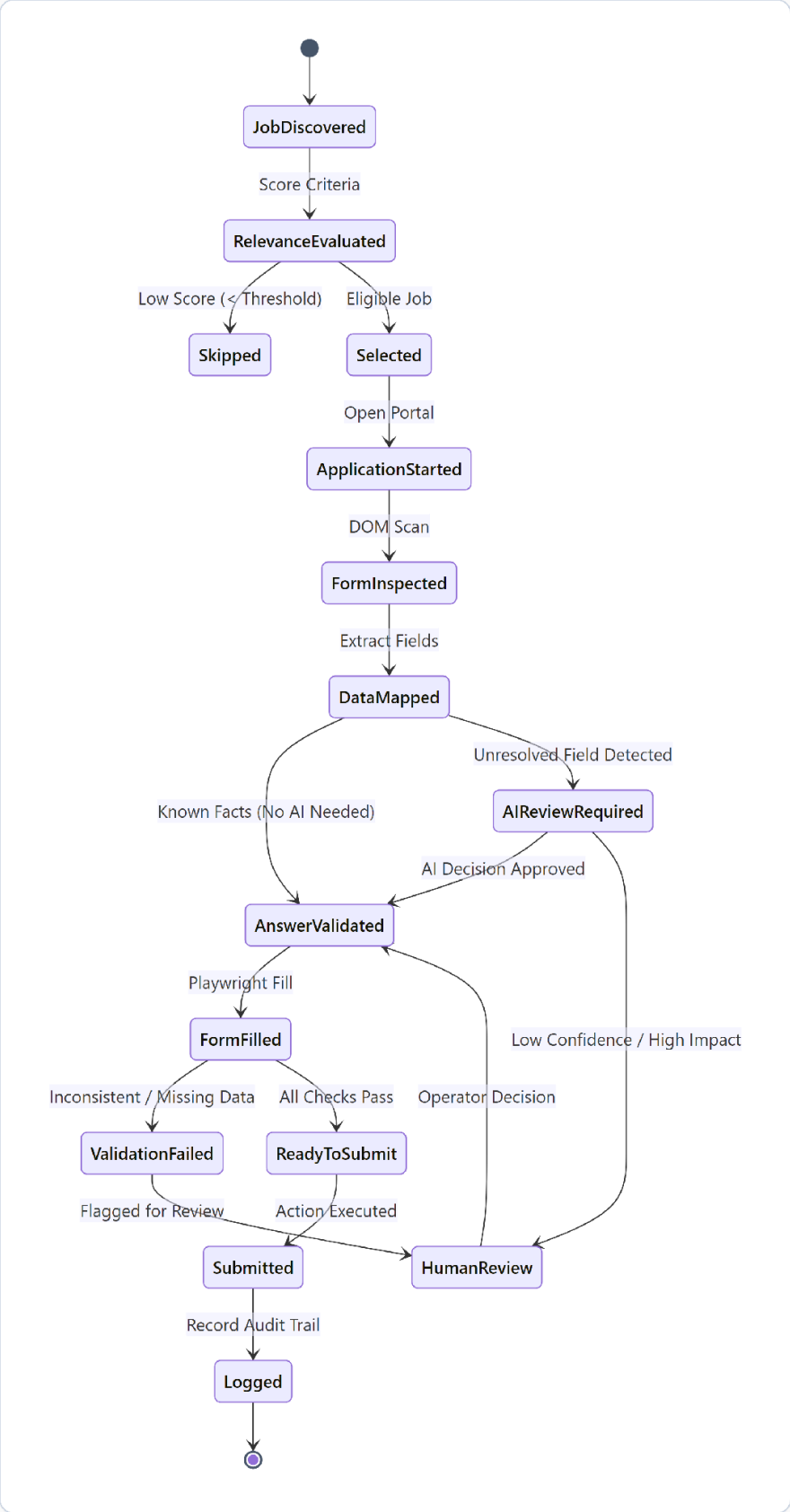


Figure 2: Candidate Application State Machine Lifecycle

6. DOM Intelligence and AI Decision Pipeline

The difficult part of the MVP is not typing into browser fields. It is understanding what the application page is asking and representing that information consistently. The DOM/Form Engine should extract semantic information from the page before the information reaches the LLM. This reduces prompt size, makes decisions testable, and separates portal HTML details from business logic.

6.1 DOM-to-Semantic Pipeline

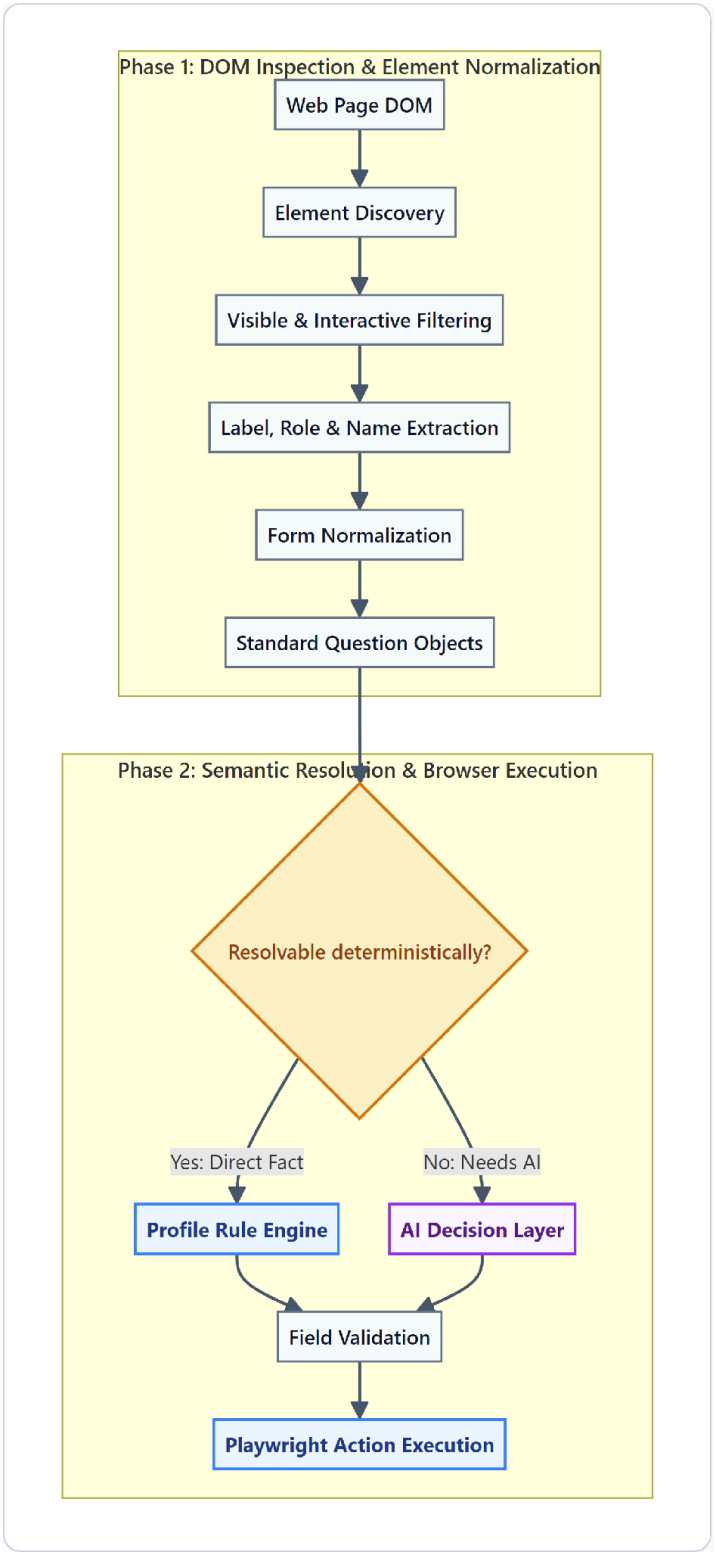


Figure 3: DOM-to-Semantic Extraction & Resolution Pipeline

6.2 Intermediate Form Representation

Each field should be converted into a predictable internal object. The browser code then operates on the representation rather than every portal-specific HTML pattern.

```
{
  "page_id": "application_step_02",
  "fields": [
    {
      "id": "field_17",
      "type": "radio",
      "label": "Do you have an MBA degree?",
      "options": ["Yes", "No"],
      "required": true,
      "source": "dom"
    }
  ]
}
```

6.3 AI Input Contract

```
{
  "candidate": {
    "education": ["MSc Computer Science"],
    "experience_years": 3,
    "skills": ["Python", "Node.js", "React"]
  },
  "job": {
    "title": "Backend Developer",
    "location": "United States",
    "requirements": ["Python", "Node.js"]
  },
  "question": {
    "type": "radio",
    "text": "Do you have an MBA degree?",
    "options": ["Yes", "No"],
    "required": true
  }
}
```

6.4 AI Output Contract

```
{
  "decision": "select",
  "answer": "No",
  "confidence": 0.98,
  "reason_code": "candidate_education_does_not_include_mba",
  "needs_human_review": false
}
```

6.5 Confidence Policy

Condition	Action
High confidence + low-impact question	Automate
Medium confidence	Automate only after validation; otherwise review
Low confidence	Human review
Candidate data missing	Human review / profile completion
Sensitive or legally consequential question	Prefer deterministic fact or human review
Unknown control	Pause and create an exception

7. Candidate Knowledge and Data Model

The candidate profile should be the authoritative source for application answers. Resume text alone is insufficient because application forms frequently ask for information not explicitly present in the resume. Resume extraction should therefore create a structured profile that the recruiter can review and correct.

7.1 Candidate Profile Structure

```
Candidate
├── Identity
│   ├── Full Name
│   ├── Email
│   ├── Phone
│   └── Location
├── Education
├── Experience
├── Skills
├── Certifications
├── Projects
├── Work Authorization
├── Sponsorship Preference
├── Job Preferences
└── Documents
```

7.2 Suggested Database Entities

Entity	Key Fields
candidates	id, name, email, phone, location, profile_json
candidate_education	candidate_id, degree, field, institution, dates
candidate_experience	candidate_id, employer, title, dates, description
candidate_skills	candidate_id, skill, proficiency
candidate_documents	candidate_id, type, storage_uri, version
jobs	id, portal, external_id, title, company, location, description, url
job_scores	job_id, candidate_id, score, decision, reasons
applications	id, candidate_id, job_id, status, timestamps
application_questions	application_id, field_id, question, type, options
ai_decisions	question_id, answer, confidence, reason_code, model
automation_logs	application_id, event_type, payload, timestamp

7.3 Data Integrity Rule

Unknown facts must remain unknown. The system must not infer a credential simply because a related skill or degree seems likely. For example, an MSc in Computer Science must not be treated as proof of an MBA, PMP, security clearance, or any other qualification.

8. Job Relevance Engine

The platform should not submit every search result. A relevance engine should create an explainable fit score based on configurable criteria. The score is a decision-support signal, not a statement of absolute suitability.

8.1 Example Scoring Framework

Dimension	Weight	Signal
Role similarity	30%	Target title and role-family match
Core skills	25%	Required skill overlap
Experience	20%	Years and seniority fit
Location/work mode	10%	USA location and preference
Education	5%	Degree alignment
Employment fit	5%	Full-time/contract/C2C preference
Other requirements	5%	Certifications/domain/tools

8.2 Example Output

```
{
  "job_id": "job_123",
  "score": 91,
  "decision": "APPLY",
  "matches": ["Python", "Node.js", "Backend Development"],
  "gaps": ["AWS certification preferred"],
  "notes": "Strong role and experience alignment."
}
```

9. Technology Stack and Implementation Approach

Layer	Recommendation	Reason
Frontend	Next.js / React	Fast dashboard development with TypeScript
Backend	Node.js + TypeScript	One runtime for API and browser orchestration
Automation	Playwright	Reliable browser control and modern web testing
Database	PostgreSQL	Relational data + JSON support
Queue	Redis + BullMQ	Useful for long-running workflows
AI	LLM behind provider abstraction	Swap models without rewriting the system
Resume parsing	PDF/text extraction service	Normalize candidate documents
Storage	S3-compatible object storage	Secure document storage
Observability	Structured logs + error tracking	Debugging and auditability

9.1 Why Playwright

Playwright is a strong fit because the MVP depends on interacting with modern web applications, handling dynamic page content, locating form elements, uploading files, navigating multi-step workflows, and capturing traces during failures. Locators should prefer stable semantic anchors such as labels, roles, and accessible names over brittle absolute XPath chains.

9.2 Where Python Can Fit

Python can be introduced as a supporting service for advanced resume parsing, NLP experiments, offline evaluation, or model benchmarking. It does not need to be the primary runtime for the MVP. A single primary stack is easier to deploy, debug, and maintain.

10. Security, Compliance, and Reliability

10.1 Candidate Data Security

- Encrypt sensitive candidate data at rest and in transit.
- Keep credentials and API keys in secure secret storage.
- Use role-based access to candidate data.
- Minimize personally identifiable information in logs.
- Use secure object storage for resumes.
- Keep version history for important candidate-profile edits.

10.2 Platform Automation Policy

The implementation must respect the selected portal's terms, permitted automation mechanisms, security controls, and access restrictions. The product must not be designed to defeat CAPTCHA, MFA, rate limits, anti-bot protections, or other controls. A triggered protection should produce a human-review state.

10.3 Reliability Strategy

- Use explicit state checks instead of fixed sleep calls wherever possible.
- Capture screenshots or trace references during development failures.
- Retry only transient network/navigation errors, not uncertain decisions.
- Persist workflow state so interrupted sessions can be resumed safely.
- Use idempotency to prevent duplicate submissions.
- Maintain regression fixtures for representative application forms.

11. Human-in-the-Loop and Exception Handling

A reliable automation system does not attempt to automate uncertainty away. Uncertainty becomes an explicit workflow state. This is particularly important in recruitment workflows because a wrong answer can create a false representation of a candidate.

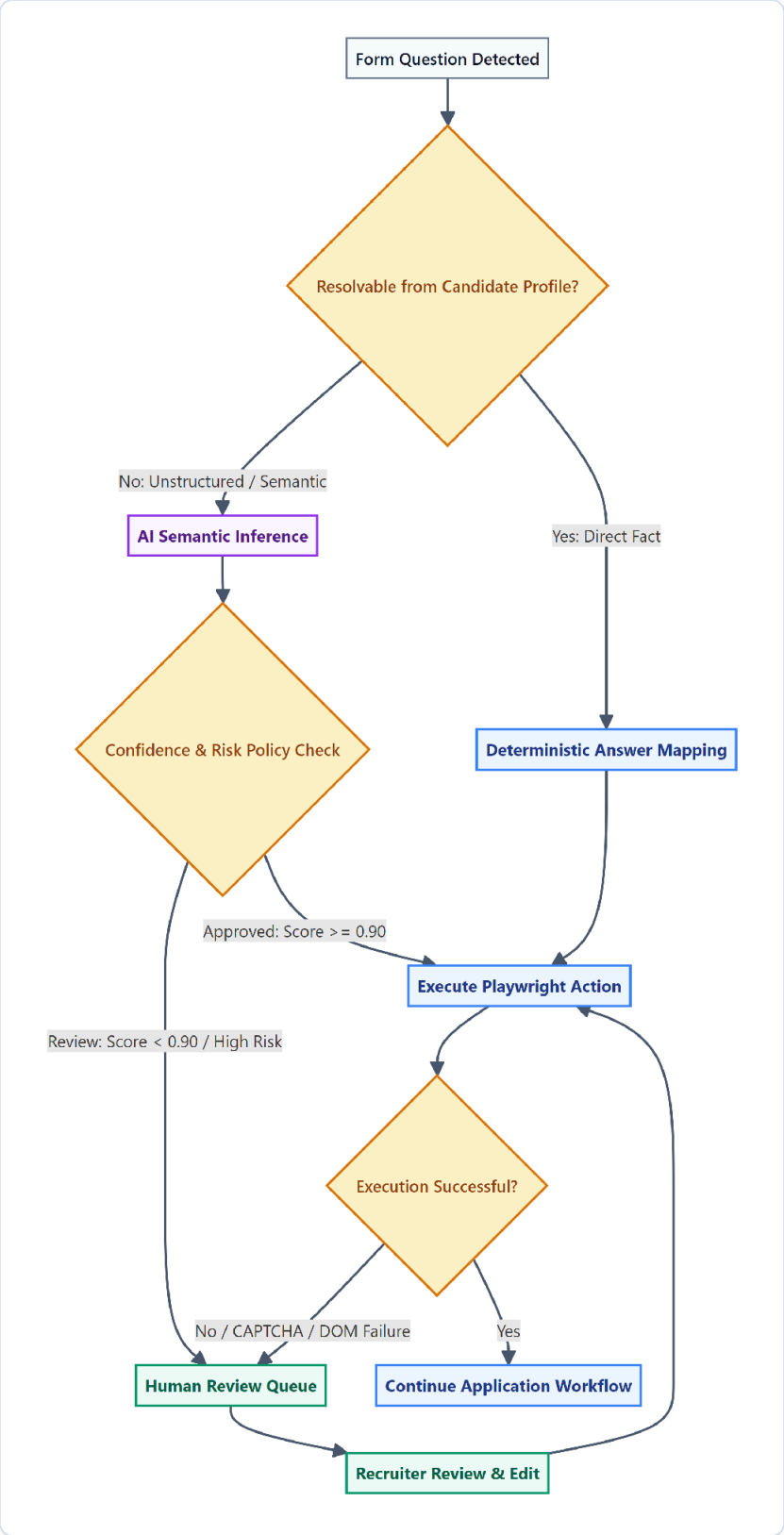


Figure 4: Human-in-the-Loop Decision Matrix & Exception Escalation

11.1 Exception Categories

Exception	Response
CAPTCHA / verification	Pause and request human action
OTP / MFA	Pause and request human action
Unknown field type	Capture evidence and pause
Missing candidate fact	Request profile completion
Low AI confidence	Human review
Contradictory candidate data	Block and resolve profile
Unexpected page	Recover or pause with error
Submission confirmation missing	Do not assume success

12. Key APIs and Internal Contracts

12.1 Example APIs

Method	Endpoint	Purpose
POST	/candidates	Create candidate profile
POST	/candidates/{id}/resume	Upload and parse resume
POST	/jobs/search	Start portal job search
POST	/jobs/{id}/score	Calculate relevance
POST	/applications	Start application workflow
GET	/applications/{id}	Read application state
POST	/applications/{id}/review	Submit human review decision
GET	/applications/{id}/logs	Read audit trail

12.2 Orchestrator Contract

```
ApplicationOrchestrator.start(applicationId)
ApplicationOrchestrator.inspectPage(applicationId)
ApplicationOrchestrator.resolveKnownFields(applicationId)
ApplicationOrchestrator.requestAIDecision(questionContext)
ApplicationOrchestrator.validateDecision(decision, formField)
ApplicationOrchestrator.executeAction(action)
ApplicationOrchestrator.validateForm(applicationId)
ApplicationOrchestrator.submit(applicationId)
ApplicationOrchestrator.recordOutcome(applicationId)
```

13. Dashboard and User Experience

13.1 Recommended Screens

- Candidate Management: create profile, upload resume, review extracted facts.
- Job Search: configure role, USA location, experience, work mode, and other filters.
- Job Results: show relevance score, matched skills, gaps, and recommendation.
- Application Workspace: display progress, current step, form state, and exceptions.
- Human Review Queue: show unresolved questions with candidate context and options.
- Application History: submitted, failed, paused, reviewed, and skipped applications.
- Audit/Logs: inspect actions, timestamps, model decisions, and failure evidence.

13.2 Example Application Workspace

```
+-----------------------------------------------------------+
| Application: Backend Developer • Example Corp             |
| Progress: Step 3 of 5                                     |
+----------------------+------------------------------------+
| Candidate Data       | Current Application Step           |
| -------------------  | ---------------------------------- |
| Name: Candidate      | Do you have an MBA?                |
| Degree: MSc CS       | [Yes]  [No]                        |
| Exp: 3 years         |                                    |
| Skills: Python...    | AI Decision: No                    |
|                      | Confidence: 0.98                   |
|                      | [Approve] [Edit] [Flag]            |
+----------------------+------------------------------------+
```

14. Development Roadmap

Phase	Focus	Deliverables
1	Discovery and feasibility	Selected portal, supported workflow map, constraints
2	Candidate system	Profile schema, resume parsing, review UI
3	Job discovery	Search, extraction, deduplication
4	Relevance engine	Rules, scoring, explanation output
5	Application engine	DOM inspection, form normalization, deterministic filling
6	AI layer	Prompt/schema, confidence, validation
7	Human review	Exception queue and resume/continue flow
8	Testing/hardening	Regression, observability, recovery
9	Pilot	Controlled internal trial

14.1 Suggested Team Roles

- Automation Engineer: Playwright, portal workflow, DOM/form engine.
- Backend Engineer: API, orchestrator, database, queues, logging.
- AI Engineer: prompts, structured outputs, relevance scoring, evaluation.
- Frontend Engineer: dashboard, application workspace, human review.
- QA Engineer: regression cases, edge cases, reliability testing.
- Team Lead/Product Owner: requirements, policy review, acceptance criteria.

15. Testing and Evaluation Strategy

15.1 Test Layers

- Unit tests for candidate mapping, scoring, schema validation, and policy rules.
- DOM normalization tests using synthetic HTML fixtures.
- AI benchmark containing representative application questions.
- Browser workflow tests for every supported application step.
- Controlled end-to-end pilot runs.
- Failure recovery tests for timeouts, network issues, unexpected pages, and partial completion.

15.2 AI Evaluation Dataset

Create a benchmark containing representative questions with the trusted candidate facts available to the model, the correct answer or acceptable answer set, question type, and expected review policy. This turns model quality into a measurable engineering target.

15.3 Important Edge Cases

- Question wording changes but meaning remains the same.
- Options appear in a different order.
- Required question has no matching candidate fact.
- Checkbox allows multiple answers.
- Textarea has a character limit.
- The next page loads dynamically.

- A field is disabled or pre-filled.
- Two questions appear to conflict.
- Submission confirmation is delayed or unclear.

16. Risks, Mitigations, and Design Trade-offs

Risk	Impact	Mitigation
Portal UI changes	High	Adapter layer, semantic locators, regression tests
AI hallucination	High	Structured output, grounding, confidence, validation
Incorrect candidate fact	High	Human-reviewed source of truth
Bot detection / blocking	High	Respect platform controls; human handoff
Long application sessions	Medium	Persist state and resume
Duplicate submission	High	Idempotency and submission-state checks
Prompt injection in page text	High	Treat page content as untrusted data
Sensitive data leakage	High	Minimize AI payload; access controls
Unsupported widgets	Medium	Pause and create exception

16.1 Prompt Injection Defense

Job descriptions and application pages are untrusted external content. The model must not interpret text found on a webpage as system instructions. System rules and trusted candidate facts should be separated clearly from page data. Requests for secrets, instructions to ignore higher-priority rules, or unrelated actions must be treated as untrusted page content.

17. Future Expansion

After the single-portal MVP is reliable, the same core can support additional portals through independent adapters. The core services should remain portal-agnostic, while search navigation, job extraction, and application navigation are implemented by portal-specific adapters.

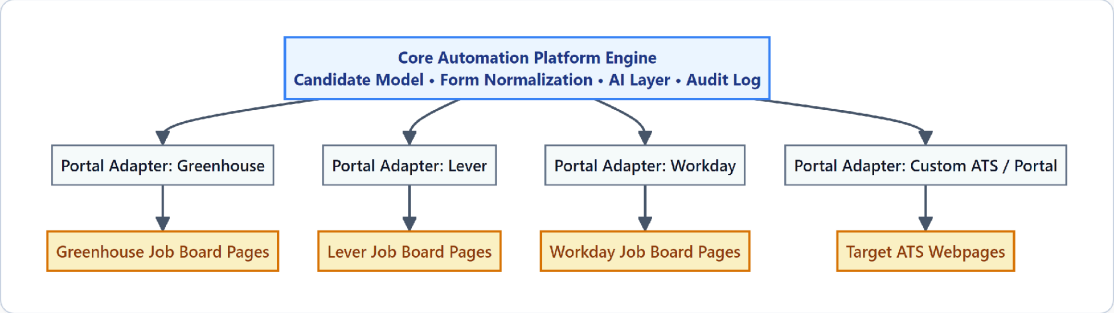


Figure 5: Modular Multi-Portal Adapter Architecture

- Additional portal adapters.
- Controlled concurrent application queues.
- Advanced resume tailoring and cover-letter generation.
- Recruiter approval before submission.
- Application-conversion analytics.
- Candidate-job matching across larger job inventories.
- Application status tracking and follow-up workflows.
- Model evaluation dashboards.

18. Final MVP Definition

The MVP is complete when a recruiter can create or select a candidate, configure a USA-focused search on one supported job portal, identify relevant jobs, open an application for a selected job, allow the system to inspect the page, map known fields from the candidate profile, use AI to resolve supported unknown questions, review exceptions when necessary, complete the form, submit the application where permitted, and see a durable application record with an auditable execution history.

The platform should be built as controlled automation rather than a blind web bot. The central pattern is: structured candidate knowledge provides facts; deterministic rules answer what can be answered safely; AI handles semantic interpretation; validation enforces correctness; Playwright performs browser actions; and humans take over when uncertainty or security controls appear.

This approach gives the team a realistic path from a focused proof of concept to a scalable multi-portal platform without forcing the MVP to solve every website and every application form on day one.

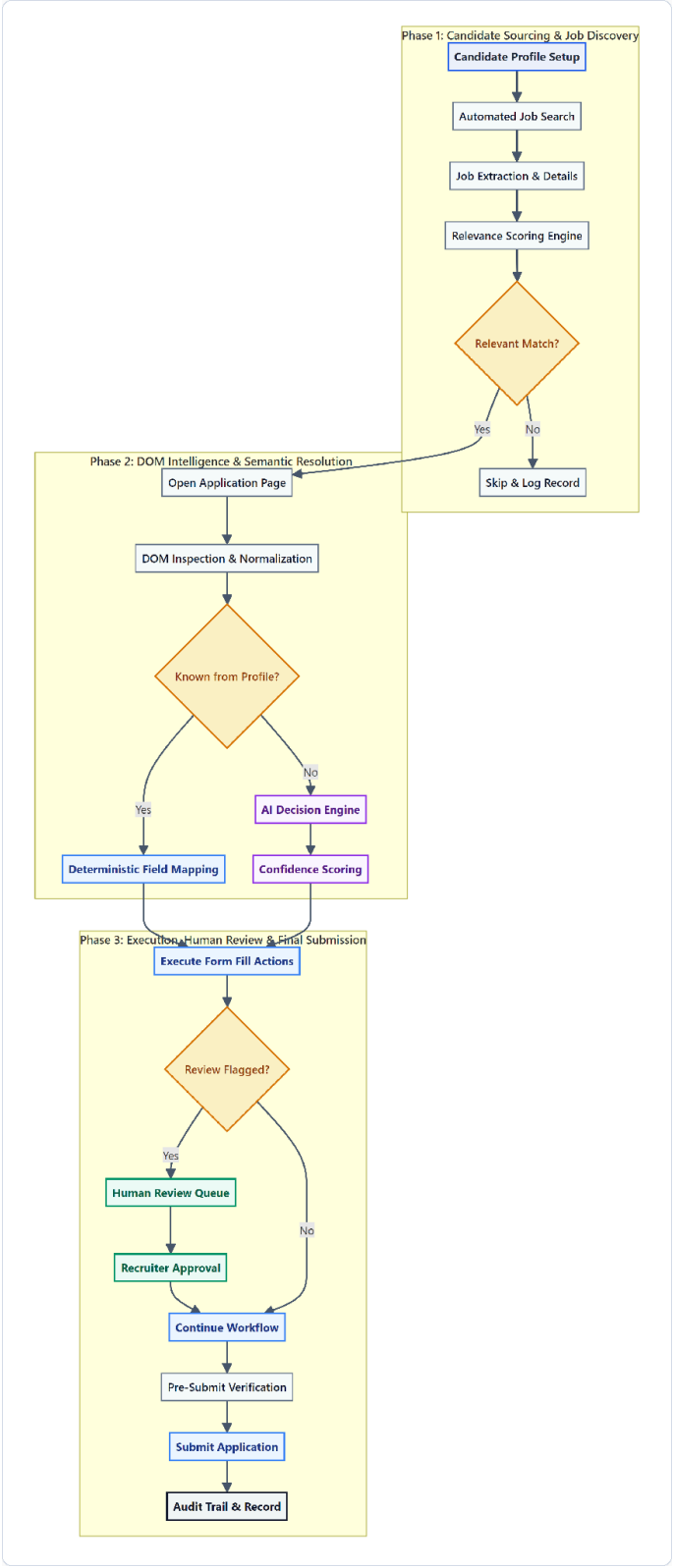


Figure A.1: End-to-End Application Automation Workflow

Appendix A.2 End-to-End Application Interaction Sequence

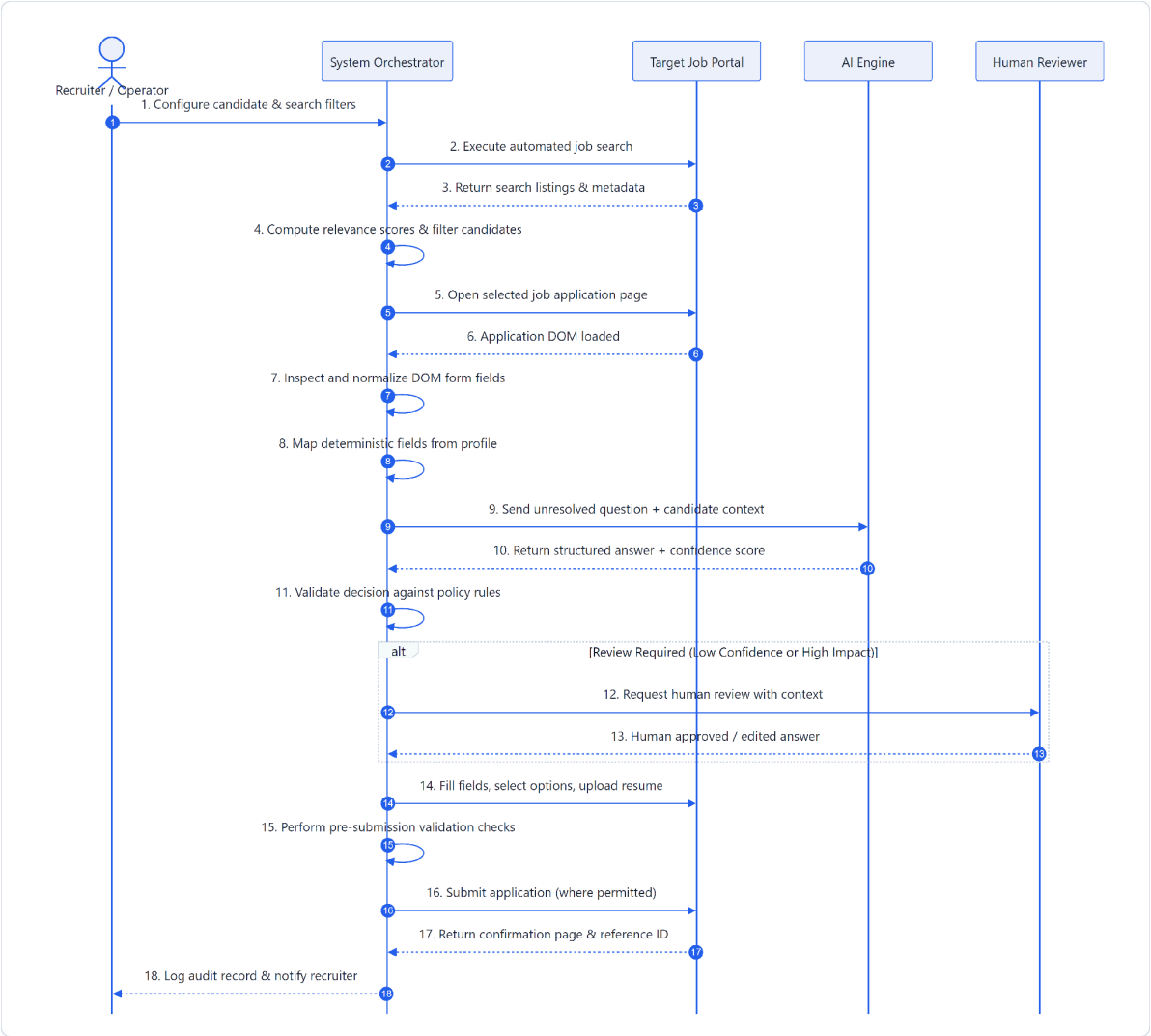


Figure A.2: End-to-End Application Interaction Sequence Diagram